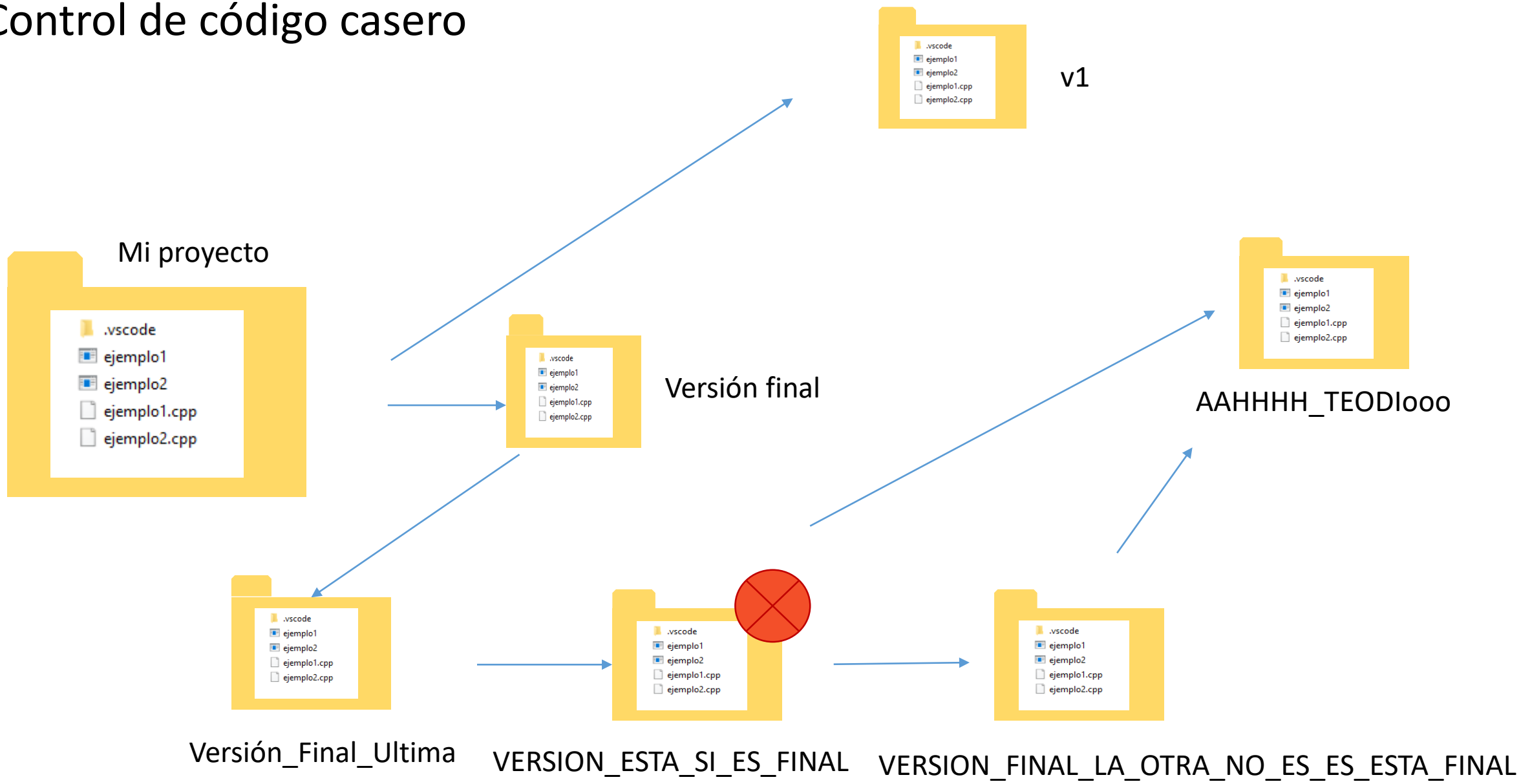
Taller de lenguajes 1

Introducción a git

¿Por qué usar git?

Un contra-Ejemplo típico

Control de código casero



¡¡Pensamos que versionar el código de forma manual funcionaría!!

¿Qué es git?

Git es un sistema de control de versiones distribuido creado por Linus Torvalds

¿Que es un sistema de control de versiones?

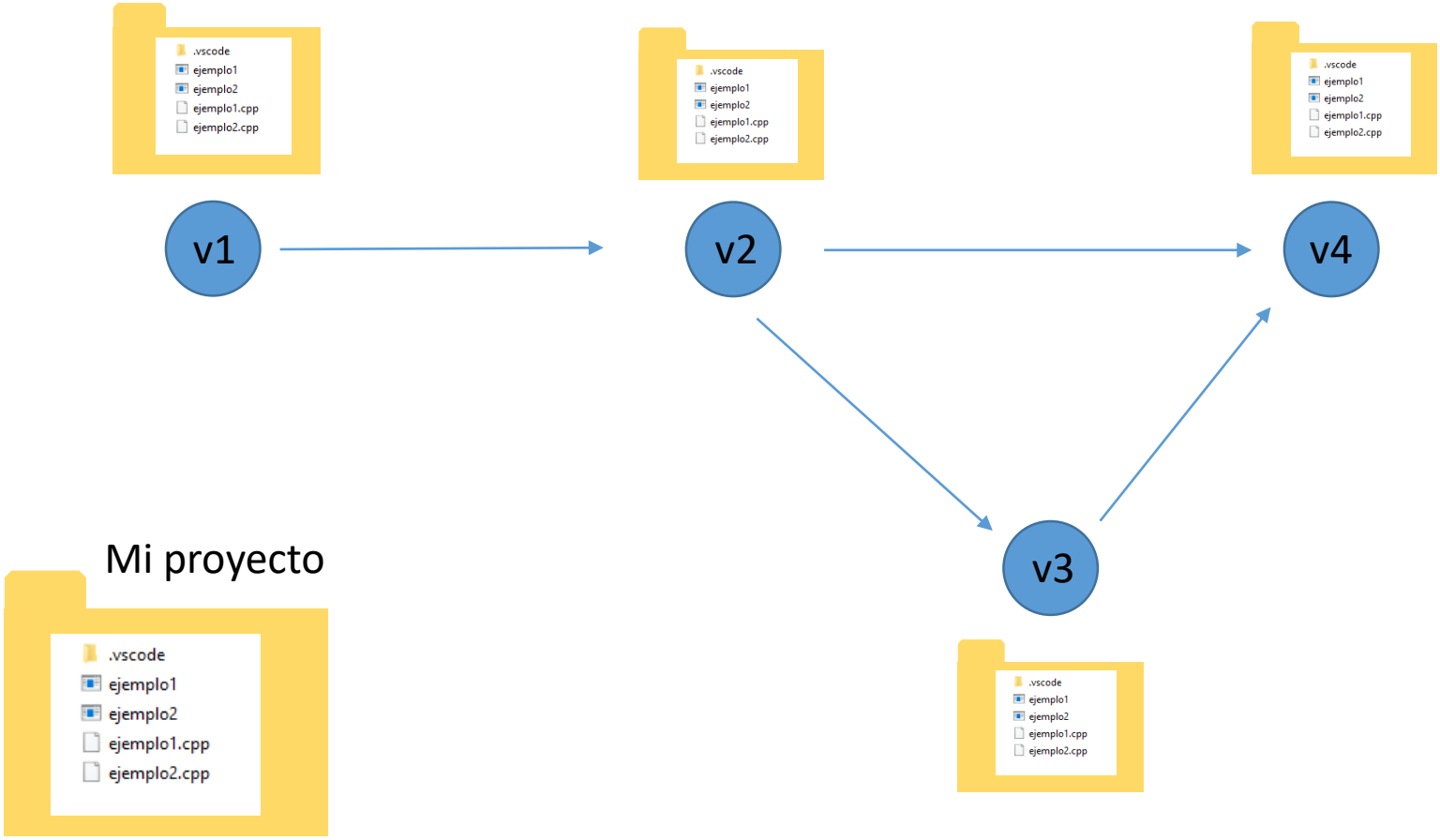
Un control de versiones es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo.

Además, git facilita el trabajo en equipo, ya que permite compartir estos cambios a través de un *Repositorio Remoto* con otras persona y qué a su vez ellos puedan compartir los suyos.

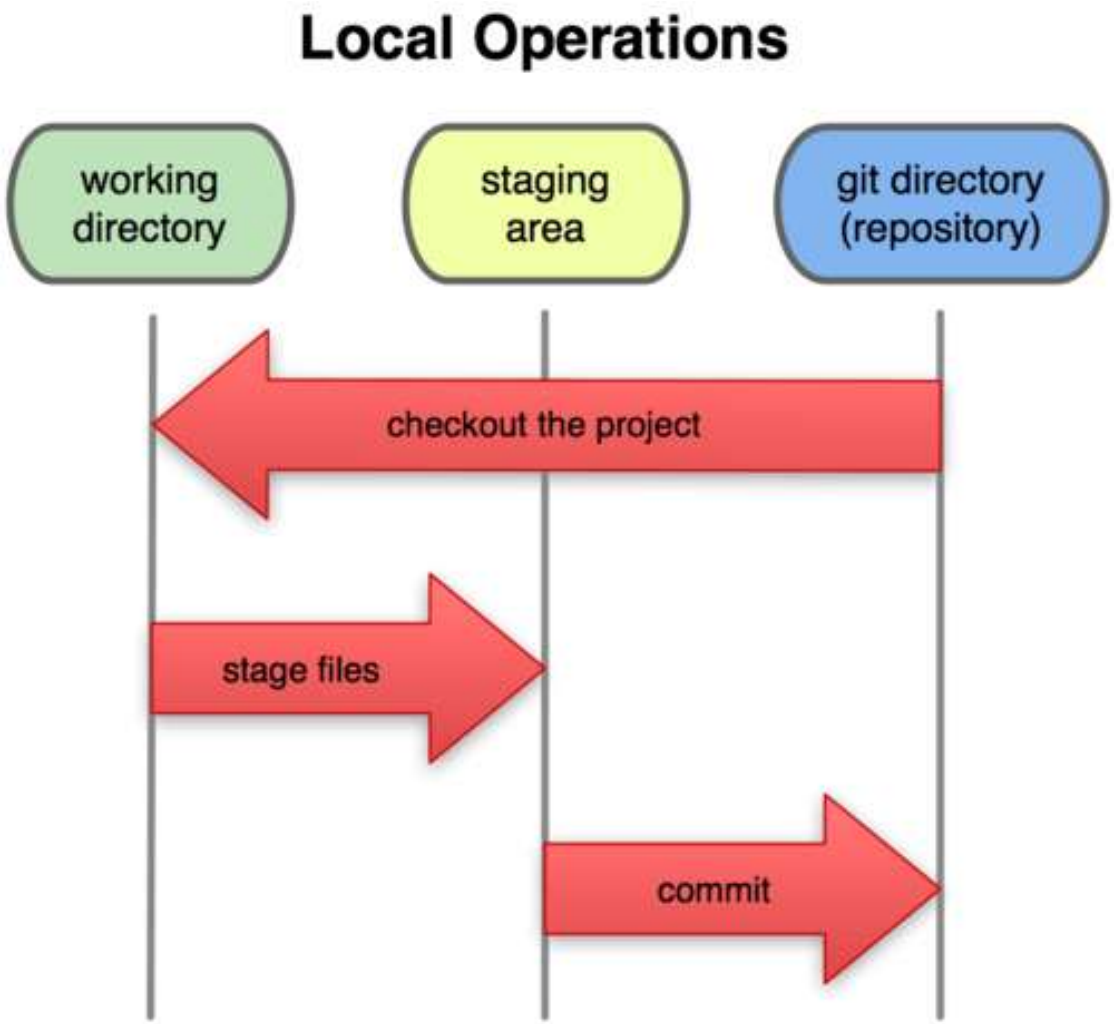
Características de git

- El diseño de Git se basó en BitKepper y en Monotone.
- Código abierto, programado en C
- Permite muchas ramas de desarrollo en paralelo (Branch)
- Útil tanto en grandes proyectos como en pequeños proyectos
- Historial de versiones
- Se puede trabajar desconectado del servidor remoto

Control de código usando git



Estados de un proyecto con git



- **Confirmado:** Los datos del archivo en el sistema de control de versiones están almacenados de manera segura en tu repositorio. (commited)

- **Modificado:** has hecho modificaciones sobre un archivo pero todavía no fueron confirmados.(untracked)

- **Preparado:** significa que se ha marcado un archivo modificado en su versión actual para que se suba al stage en la próxima confirmación. (staged)

Ejercicio paso a paso

Crear un repo y agregar un archivo al repositorio

TAREA Abrimos la consola de git o la terminal de windows

Comando > git init // inicio el repositorio

Mensaje de salida de consola Initialized empty Git repository in C:/Repositorio/Clase de git/tp/.git/

TAREA Creamos un archivo de texto Nuevo.Txt

Comando > git add . // Agrego los cambios al staging

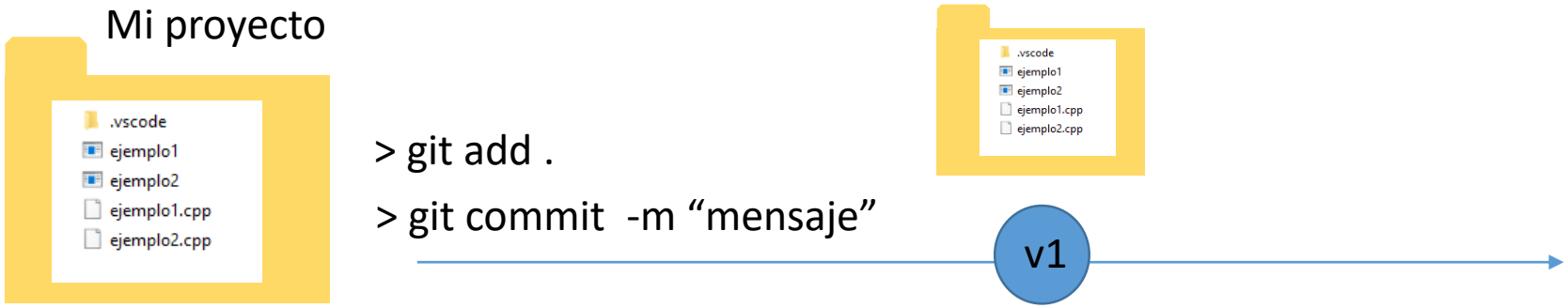
Comando > git commit -m “commit inicial” // Agrego los cambios al repositorio LOCAL

Mensaje de salida de consola>

```
[master (root-commit) f0e82b9] commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Nuevo.txt
```

TAREA Agregamos el texto “Hola mundo” en el archivo Nuevo.Txt

Control de código usando git



Ejercicio paso a paso

Agregando cambio al archivo

TAREA Agregamos el texto “Hola mundo” en el archivo Nuevo.Txt

Comando > git status // Pido el estado del repositorio en este momento

Mensaje de consola>

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Nuevo.txt
no changes added to commit (use "git add" and/or "git commit -a")
```

// Nos informa que hay cambios sin su respectivo commit

Comando > git add . // Agrego los cambios al staging

Comando > git commit -m “segundo commit” // Agrego los cambios al repositorio LOCAL

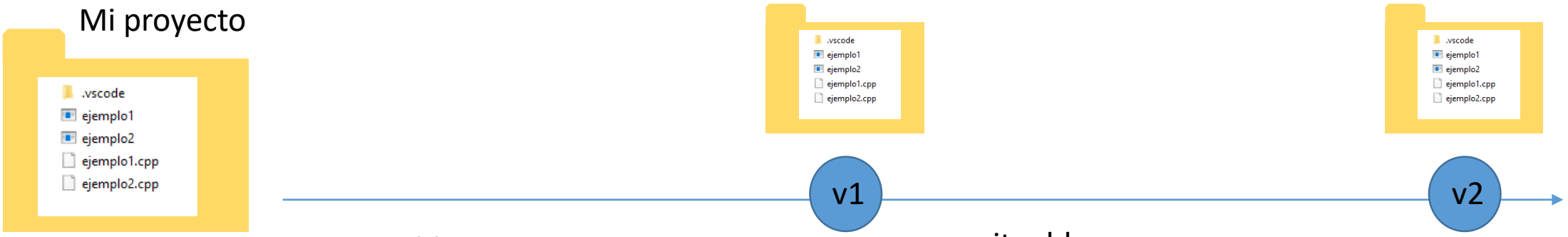
Comando > git status // Pido el estado del repositorio en este momento

Mensaje de consola>

```
On branch master
nothing to commit, working tree clean
```

// Nos informa que no hay cambios para “commitear”

Control de código usando git



> git add .

> git commit -m “commit inicial”

> git add .

> git commit -m “segundo commit”

Ejercicio paso a paso

Navegando los commits realizados

Comando

```
tp> git log
```

// permite visualizar el historial de los commit

Mensaje de consola>

historial de los commits

```
commit 139c097cefe13c94c075b4f434a26696b51283a6 (HEAD -> master)
Author: javier <jgrana@herrera.unt.edu.ar>
Date:   Thu Mar 21 11:42:31 2024 -0300

    segundo commit

commit f0e82b95c2cd5a03de34867080113d58a7130cdc
Author: javier <jgrana@herrera.unt.edu.ar>
Date:   Thu Mar 21 11:26:07 2024 -0300

    commit
```

//ID de commit + Head/Branch
//Autor del commit
//Fecha de commit
//Mensaje del commit

Comando

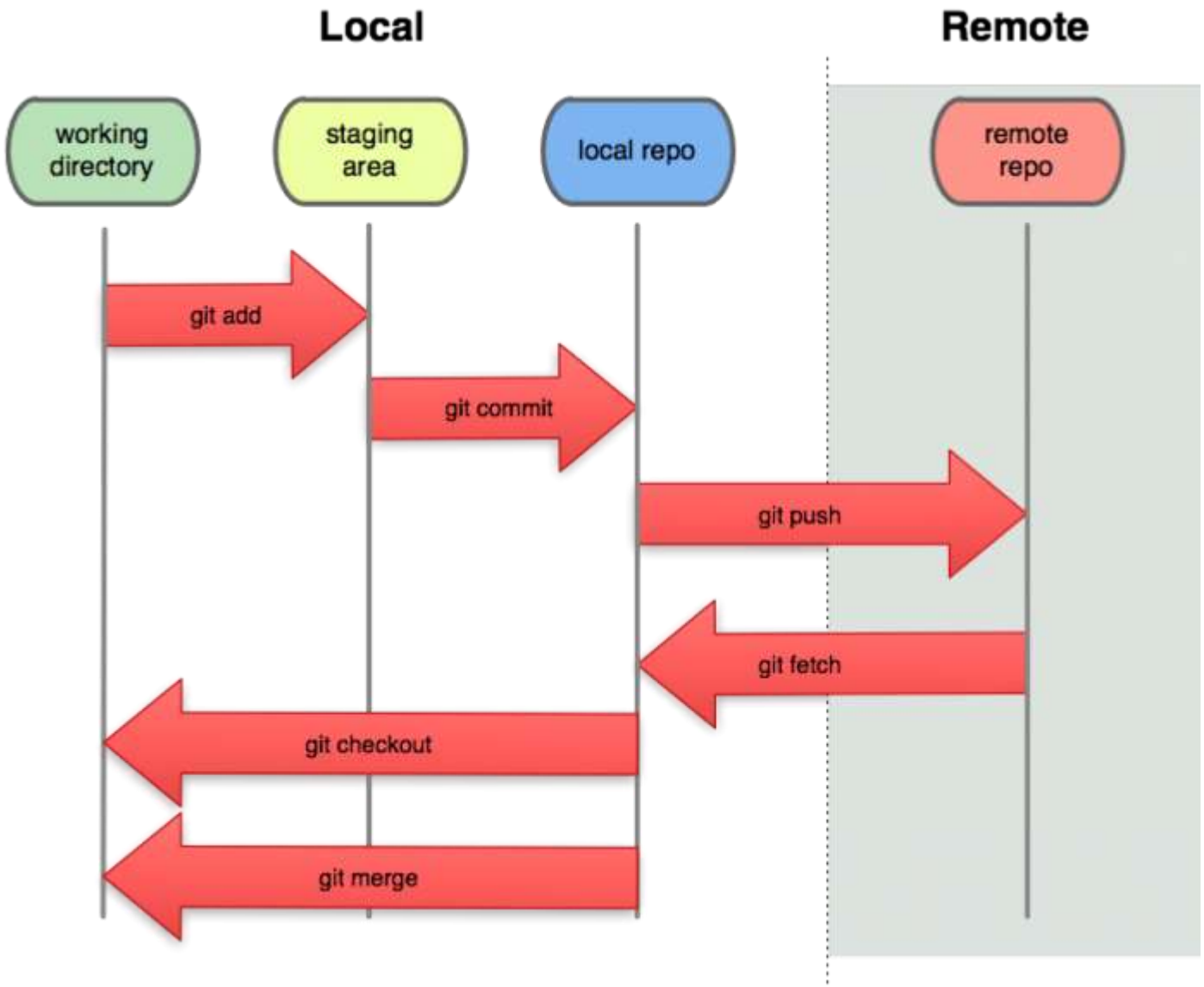
```
git log --oneline
```

Mensaje de consola>

```
139c097 (HEAD -> master) segundo commit
f0e82b9 commit
```

// ID de commit + Head/Branch + mensaje
// ID de commit + mensaje

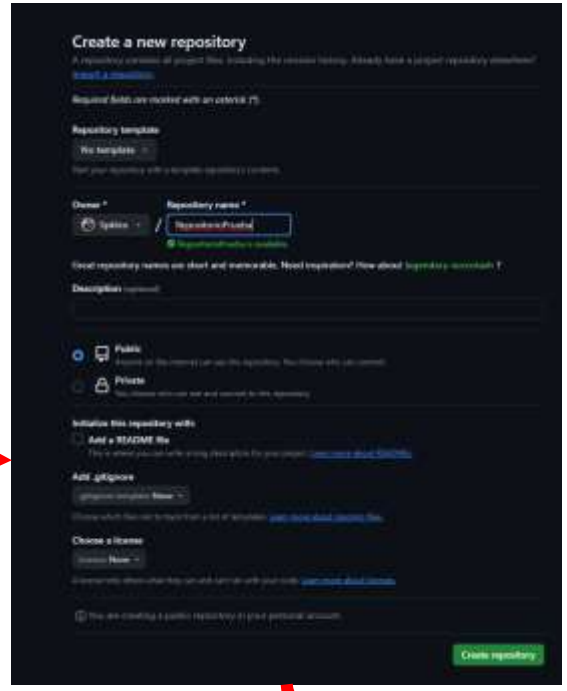
Estados de un proyecto con git y un repositorio remoto



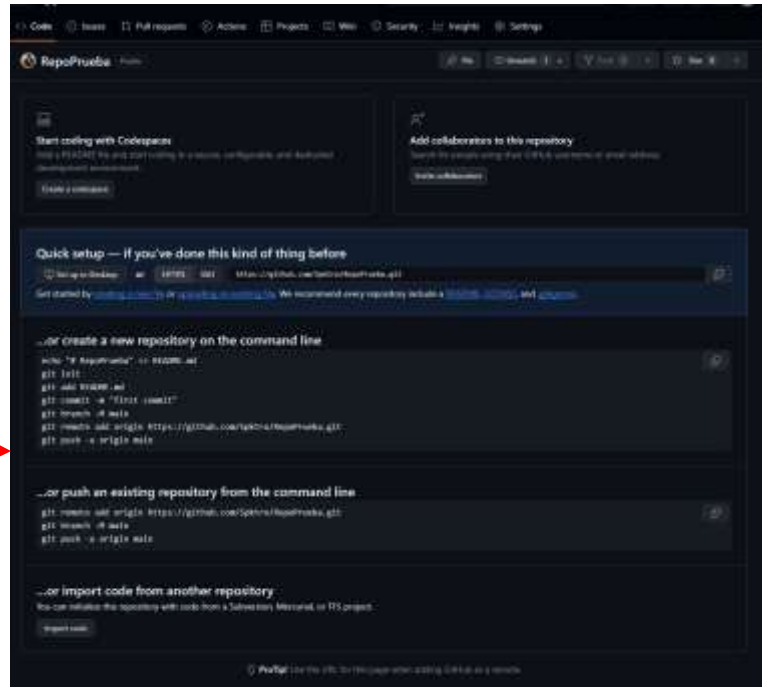
Ejercicio paso a paso

Subiendo los commits al repositorio remoto

Nos registramos y autenticamos en github



Creamos un repositorio remoto



Ejercicio paso a paso

Subiendo los commits al repositorio remoto

Comando > git remote -v // muestra todos los repositorios remotos

Mensaje de consola> // -mensaje vacio-

Comando > git remote add origin https://github.com/Spktro/RepoPrueba.git // agrega un repositorio remoto

Comando > git remote -v // permite ver todos los repositorios remotos

Mensaje de consola>
```
origin  https://github.com/Spktro/RepoPrueba.git (fetch)
origin  https://github.com/Spktro/RepoPrueba.git (push)
```

Comando >git push --set-upstream origin master // agrega los commits y los branchs locales al repositorio remoto

Mensaje de consola>
```
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (6/6), 427 bytes | 427.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Spktro/RepoPrueba.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Ejercicio paso a paso

Clonando repositorio remoto

TAREA Crear una carpeta donde vamos a clonar el repositorio y posicionarnos ahí

Comando > git clone https://github.com/Spktro/RepoPrueba.git // clonamos el repositorio remoto de forma local

Mensaje de consola>

```
Cloning into 'RepoPrueba'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 6 (delta 0), reused 6 (delta 0), pack-reused 0
Receiving objects: 100% (6/6), done.
```

Comandos útiles

git fetch:
Descarga los cambios realizados en el repositorio remoto.

git commit -am "<mensaje>":
Confirma los cambios realizados.
El “mensaje” generalmente se usa para asociar al *commit* una breve descripción de los cambios realizados.

git push origin *<nombre_rama>*:
Sube la rama “nombre_rama” al servidor remoto.

git status:
Muestra el estado actual de la rama, como los cambios que hay sin commitear.

git add *<nombre_archivo>*:
Comienza a trackear el archivo “nombre_archivo” si se coloca el carácter “.” (punto) entonces subirá todo lo que haya cambiado

git push origin *<nombre_rama>*:
Commitea los cambios desde el branch local origin al branch “nombre_rama”.

git remote prune origin:
Actualiza tu repositorio remoto en caso que algún otro desarrollador haya eliminado alguna rama remota.

git reset --hard HEAD:
Elimina los cambios realizados que aún no se hayan hecho *commit*.

git revert *<hash_commit>*:
Revierte el *commit* realizado, identificado por el “hash_comm”.it

Cconfigurando git